

1. SUPERVISED LEARNING:

ALGORITHMS : 1.Regression 2.Classification

1.REGRESSION ALGORITHM: Used when the output is a number.

Regression Algorithm includes ---> Linear Regression ---> Multiple Regression ---> Polynomial Regression
---> Decision Tree Regression ---> Random Forest Regression ---> Support Vector Regression

2.CLASSIFICATION ALGORITHM: Used when the output is a category/class.

Classification Algorithm includes ---> Logistic Regression ---> Decision Tree ---> Random Forest ---> KNN ---> SVM

--->Linear Regression - Finds a straight-line relationship between input and output.

Predicts continuous values.

"Straight line formula ($y=mx+b$)" y = output (vertical value) x = input (horizontal value) m = slope(weight) b = intercept (bias)

MSE (Mean Squared Error)

why we need cost function?

A cost function tells the machine learning model how wrong it is, so the model can improve by minimizing that error.

Mean Squared Error (MSE): -- Most common cost function for linear regression is MSE MSE is a loss function that measures how wrong the predictions are. FORMULA: $MSE = error^2 + error^2 +$

error^2/n error=Actual value - Predicted Value STEPS: Step 1: Find the Error Step 2: Square the Errors
Step 3: Add All Squared Errors Step 4: Divide by Number of Values

MSE = Measures the error.

Gradient Descent

Gradient Descent is an optimization algorithm. Its job is to minimize the MSE (or another loss function).

Steps: Step 1: Initialize Parameters Step 2: Make Predictions Step 3: Calculate the Cost (Error) Step 4: Compute the Gradient
Step 5: Update Parameters Step 6: Repeat

Gradient Descent = Minimizes the error.

```
In [2]: import numpy as np
# Dataset
x = np.array([1,2,3,4])
y = np.array([3,5,7,9])
# Initial Values
m = 0 # Start with random guesses
b = 0
learning_rate = 0.01 # Controls how much the model updates each step
epochs = 1000 # Number of iterations
n = len(x)
for i in range(epochs):
    # Predictions
    y_pred = m * x + b # Current Line equation
    # Gradients
    dm = (-2/n)*np.sum(x*(y-y_pred)) # how much to change slope
    db = (-2/n)*np.sum(y-y_pred) # how much to change intercept
    # Update parameters
    m = m - learning_rate * dm
    b = b - learning_rate * db # Move toward lower error.
print("Slope (m):",m)
print("Intercept (b):",b)
```

Slope (m): 2.0048610156782556
Intercept (b): 0.9857080211211781

OLS (Ordinary Least Squares)

is a method used in Linear Regression to find the best-fit line. Its goal is to find the values of the weights and bias that minimize the sum of squared errors. STEPS:

1. Squares each error.
2. Adds them together.
3. Finds the line with the smallest total squared error.

```
In [13]: import pandas as pd
from sklearn.linear_model import LinearRegression
# Sample Data
df = pd.DataFrame({'Experience':[1,2,3,4,5], 'Salary': [30000,35000,40000,45000,50000]})
# Input Feature
x = df[['Experience']]
# Target Variable (y)
y = df['Salary']
# Create Model
model = LinearRegression()
# Train Model (OLS happens here)
model.fit(x,y)
# Predict Salary
predictions = model.predict(x)
# Results
print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)
print("Predicted Salaries:",predictions)
# When fit() runs, OLS automatically calculates the best slope
```

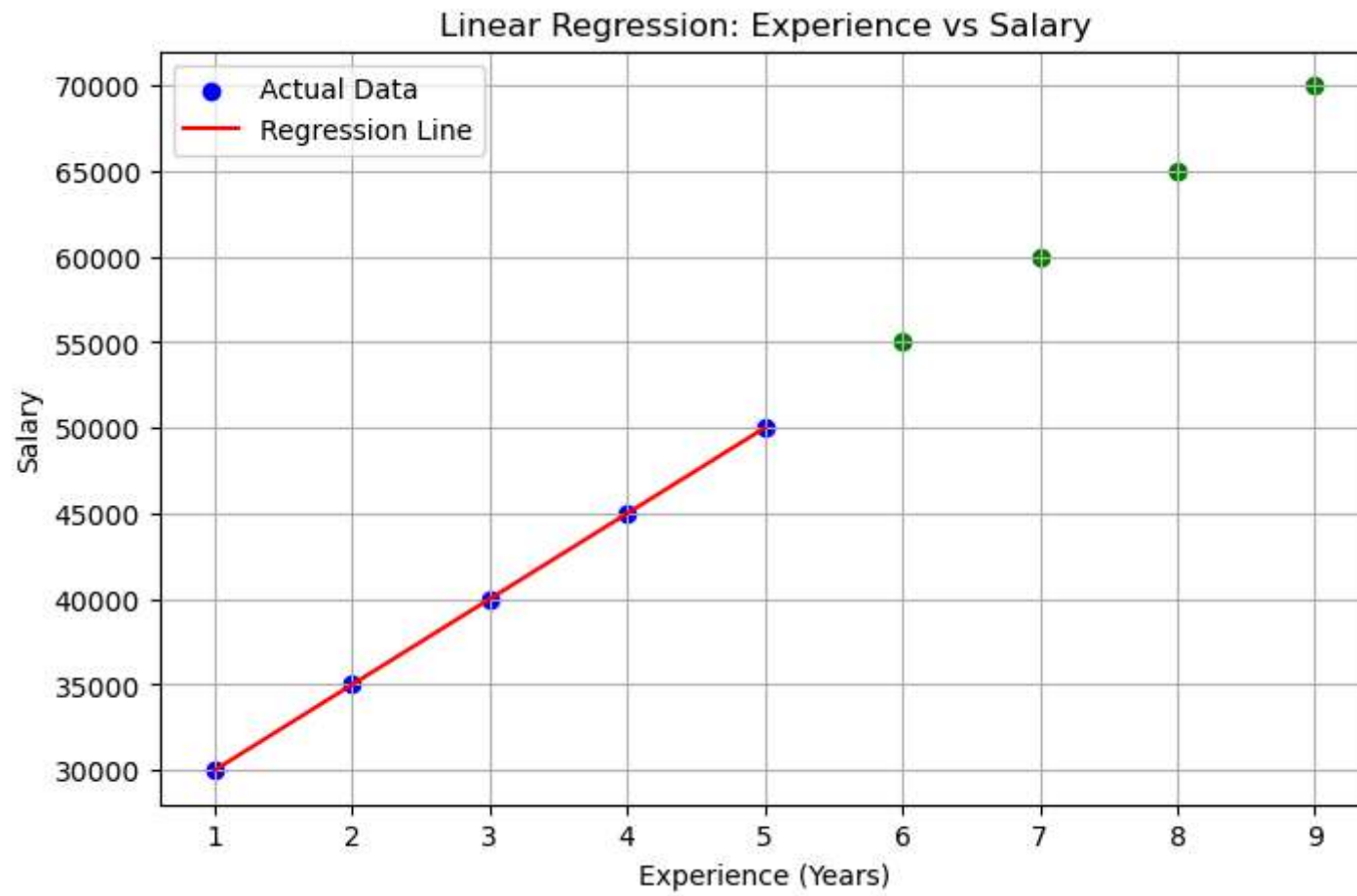
Slope: 5000.000000000001
Intercept: 24999.999999999996
Predicted Salaries: [30000. 35000. 40000. 45000. 50000.]

```
In [16]: new_experience=pd.DataFrame({'Experience': [6,7,8,9]})
#vPredict salary for new values
new_predictions=model.predict(new_experience)
```

```
print("Predicted Salaries for 6,7,8,9:")  
print(new_predictions)
```

Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]

```
In [17]: import matplotlib.pyplot as plt  
  
plt.figure(figsize=(8,5))  
  
# Original data points  
  
plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual Data')  
  
# Regression Line (trained data)  
  
plt.plot(df['Experience'], predictions, color='red', label='Regression Line')  
  
# New predictions  
  
plt.scatter(new_experience['Experience'], new_predictions, color='green')  
  
# Labels  
  
plt.xlabel("Experience (Years)")  
  
plt.ylabel("Salary")  
  
plt.title("Linear Regression: Experience vs Salary")  
  
plt.legend()  
  
plt.grid()  
  
plt.show()
```



MULTIPLE REGRESSION

In []: uses more than one independent variable (feature) to predict a dependent variable (target).

Example: Predicting AQI using:

1. Temperature
2. Humidity
3. Wind Speed

Inputs (X): Temperature, Humidity, Wind Speed

Output (y): AQI

```
In [21]: ##Multiple Regression Code

import pandas as pd

from sklearn.linear_model import LinearRegression

# Dataset

df = pd.DataFrame({

    'Area1': [1000, 1200, 1500, 1800, 2000],
    'Area2': [3000, 4000, 5000, 36878, 7390],
    'Bedroom1': [2, 3, 3, 4, 4],
    'Bedroom2': [3, 4, 8, 5, 9],
    'Age1': [10, 8, 5, 4, 2],
    'Age2': [8, 19, 21, 39, 90],
    'Price1': [300000, 350000, 450000, 500000, 600000],
    'Price2': [200000, 100000, 303000, 280000, 450000]
})

# Features

X = df[['Area1', 'Area2', 'Bedroom1', 'Bedroom2', 'Age1', 'Age2']] #Using multiple features.

# Target

y = df['Price2']

# Model

model = LinearRegression()

# Train

model.fit(X, y) #Learns the relationship between all features and price.

# Prediction

pred = model.predict(X) #predict house price

print("Coefficients:", model.coef_)
```

```
print("Intercept:", model.intercept_)
```

```
Coefficients: [-4.88752000e+02  1.56588275e+01 -1.21568462e+05  1.12987582e+05  
              4.41091034e+04  7.17187816e+03]
```

```
Intercept: 47483.637670469994
```

POLYNOMIAL REGRESSION

```
In [ ]: ##### 3. Polynomial Regression
- Polynomial Regression is an extension of Linear Regression that captures non-linear relationships by adding polynor
##### Why polynomial regression
- Not all relationships are straight lines
##### Degree Selection
Degree determines model complexity
- low degree
  1. Simple model
  2. May underfit
- high degree
  1. more flexible
  2. may overfit
- optimal degree
  1. Best balance
  2. good generalization on new data
##### Advantages
- Captures non-linear patterns
- More flexible than linear regression
##### Disadvantages
- Sensitive to outliers
- Higher risk of overfitting
- Harder to interpret
```

```
In [22]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
# Sample dataset
df = pd.DataFrame({

    'Hours': [1, 2, 3, 4, 5],
```

```

    'Marks': [10, 25, 45, 70, 100]

})
X = df[['Hours']]
y = df['Marks']

# Polynomial transformation
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)

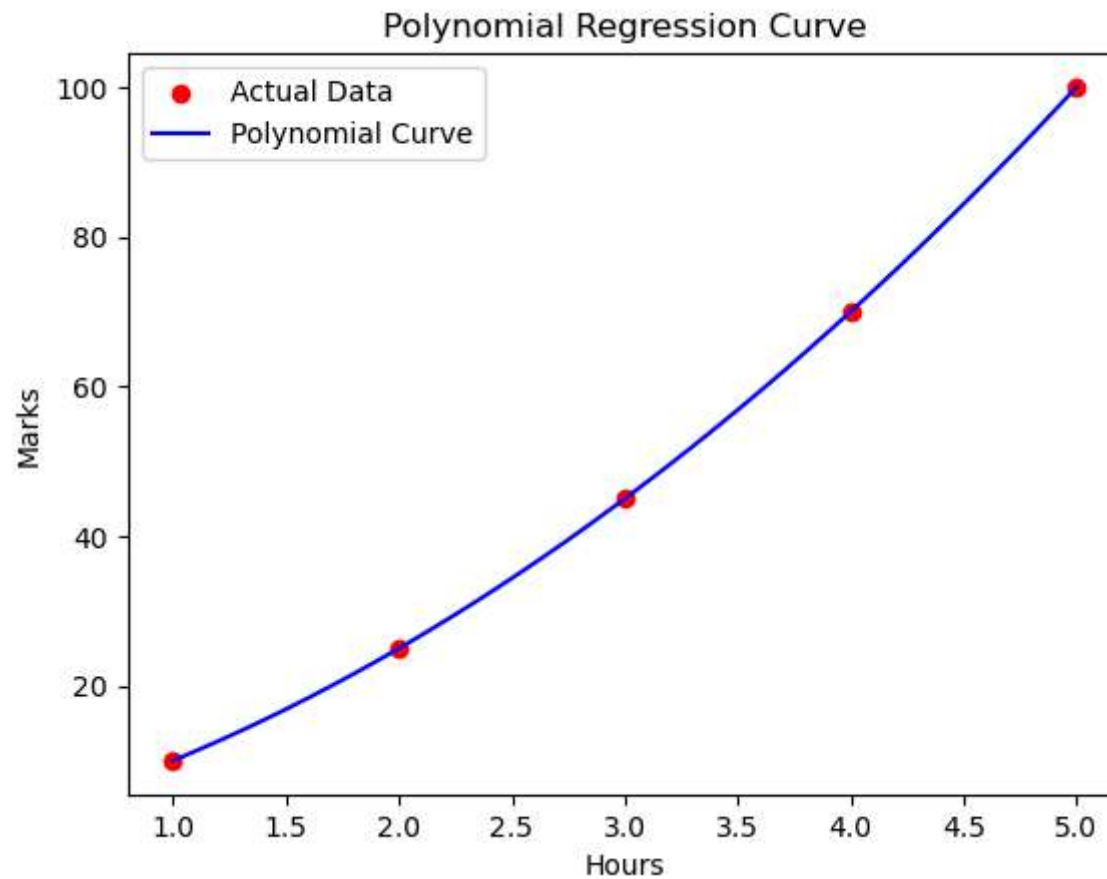
# Model training
model = LinearRegression()
model.fit(X_poly, y)

# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)

# Predictions for curve
y_curve = model.predict(X_range_poly)

# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()

```



```
In [ ]: kuick.io
        shortcut for sharing

import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)

y = np.array([2,5,8,15,28,35,50,60,85,95])
```

```

# Degree 1 - Underfitting-Straight Line misses the pattern

poly1 = PolynomialFeatures(degree=1)

X1 = poly1.fit_transform(X)

# Degree 4 - Good fit_Capture the overall curve

poly4 = PolynomialFeatures(degree=4)

X4 = poly4.fit_transform(X)

# Degree 9 - Overfitting-tries to pass through every point-Learns noise instead of pattern

poly9 = PolynomialFeatures(degree=9)

X9 = poly9.fit_transform(X)

model1 = LinearRegression().fit(X1,y)

model2 = LinearRegression().fit(X4,y)

model9 = LinearRegression().fit(X9,y)

plt.scatter(X,y)

plt.plot(X, model1.predict(X1), label="Degree 1")

plt.plot(X, model2.predict(X2), label="Degree 2")

plt.plot(X, model9.predict(X9), label="Degree 9")

plt.legend()

plt.show()

```

REGULARIZATION

In []: It is a way to trade a little training accuracy for better performance on new data.

```
>>Regularization Types:
  1.Ridge Regression  (L2 Regularization)
  >> Adds the sum of the squared weights:
  >> Shrinks weights toward zero.
  >> Most common form of regularization.

  2.Lasso Regression  (L1 Regularization)
  >> Adds the sum of the absolute values of the weights
  >> Can drive some weights to exactly zero.
  >> Useful for feature selection.

  3. Elastic Net
  >> Combines L1 and L2 regularization.
```

In [23]: *##Ridge Code*

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=5,
    noise=
    random_state=42
)
ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)
```

Coefficients:

```
[62.4844981  98.07649112 57.01070157 52.31272567 35.35343002]
```

In [24]: *##Lasso Code*

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
```

```

    n_features=10,
    noise=20,
    random_state=42
)
lasso = Lasso(alpha=2.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)

```

Coefficients:

```

[17.88753444 54.76076704  0.          60.9632585  90.0800089  68.30461063
 81.35787563  4.03519677  2.86610797 67.99906953]

```

In []: *##ElasticNet Code*

```

from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
elastic = ElasticNet(
    alpha=3.0,
    l1_ratio=0.5
)
elastic.fit(X, y)
print("Coefficients:")
print(elastic.coef_)

```